QA ENABLEMENT TRACK

Accessibility for QA Engineers

Test the web the way real people use it — with keyboards, screen readers, magnifiers, and a standard to hold it to.

From first principles → WCAG 2.2 AA → ARIA → the test process you'll actually run.

WHAT WE'LL COVER

From empathy to evidence

Foundations

Why a11y matters · disability as a spectrum · assistive tech

Screen readers

NVDA, JAWS, VoiceOver · browse vs forms mode · live demo

WCAG 2.2

Structure, conformance, and all 55 Level A/AA criteria

The test process

Automated → manual → screen-reader, and why all three

ARIA done right

Roles, states, the 5 rules, and when NOT to use it

Reporting & tooling

Issue format · AI assists · checklists · references

01

FOUNDATIONS

Why this matters

Accessibility isn't a feature you add. It's whether people can use what you ship — at all.

FOUNDATIONS

What is accessibility?

Designing and building so that people with disabilities can perceive, understand, navigate, and interact with digital products — and contribute back.

Perceive — content reaches every sense.

Operate — works by keyboard, touch, voice, switch.

Understand — clear, predictable, forgiving.

Robust — works with assistive tech, now and later.

The QA reframe

A bug that blocks a screen-reader user is still a blocking bug. “Works on my machine” isn't done — it's untested on half your inputs.

~16% of people live with a significant disability — your largest ‘minority’ user group.

FOUNDATIONS

Four reasons it's non-negotiable

It's the law

ADA, Section 508, the European Accessibility Act and EN 301 549 all point at WCAG. Lawsuits target the gaps QA should have caught.

It's reach

Disabled users + their networks are a trillion-dollar market. Inaccessible = locked-out customers.

It's quality

Captions, clear labels, and keyboard support help everyone — the curb-cut effect.

It's craft

Semantic, well-tested UI is more robust, more testable, and ages better.

WARM-UP · 2 MIN

Myth or Fact?

MYTH

"Accessibility is just alt text."

Alt text is one of 55 AA criteria. Keyboard, focus, contrast, and semantics matter just as much.

MYTH

"If automated tools pass, we're compliant."

Tools catch ~30–40%. The rest is manual + screen-reader judgement.

FACT

"Accessible design helps non-disabled users too."

Captions in loud rooms, big tap targets on the bus — the curb-cut effect is real.

FOUNDATIONS

Four broad categories — one wide spectrum

Visual

Blindness, low vision, color blindness

Screen readers, magnifiers, high contrast

Auditory

Deafness, hard of hearing

Captions, transcripts, sign language

Motor

Tremor, paralysis, limb difference, RSI

Keyboard, switches, voice, eye-tracking

Cognitive

ADHD, dyslexia, autism, memory

Clear language, structure, low distraction

Disability is a spectrum, not a switch

Permanent
one arm · deaf · blind

Temporary
broken arm · ear infection · eye dilation

Situational
new parent holding a baby · loud bar · bright sunlight

FOUNDATIONS

Assistive technology by disability

Assistive technology	Who relies on it	What QA should remember
Screen readers	Blind / low vision / some cognitive	Reads UI aloud from the accessibility tree
Refreshable braille display	Deafblind / blind	Renders text as tactile braille cells
Screen magnifiers / zoom	Low vision	Enlarges a region; reflow & contrast matter
Captions & transcripts	Deaf / hard of hearing	Text equivalent of audio & video
Voice control	Motor / RSI	“Click Search” — needs label-in-name
Switch access	Severe motor	1–2 inputs scan & select; needs logical focus order
Eye-tracking / head wand / mouth stick	Severe motor	Point without hands; needs large targets

02

SCREEN READERS

How blind users hear the web

Your most revealing test tool. If it makes sense by ear, it usually works for everyone.

SCREEN READERS

The three you'll test with

NVDA

Windows · Free

Best paired browser

Firefox / Chrome

The QA default — free, scriptable, what most testers use.

JAWS

Windows · Paid

Best paired browser

Chrome / Edge

Enterprise standard; many real users. Test if your audience uses it.

VoiceOver

macOS & iOS · Built-in

Best paired browser

Safari

The only practical way to test the Apple ecosystem.

Rule of thumb: test each screen reader with its best-paired browser — pairings expose different bugs.

SCREEN READERS

What it actually reads

Your HTML + ARIA

Roles, names, states

Accessibility tree

Browser builds a semantic model

Speech / braille

User hears name + role + state

IT ANNOUNCES THREE THINGS

Name — “Email address” (the label)

Role — “edit text” (it's an input)

State/Value — “required, invalid”

```
<label for="em">Email address</label>
<input id="em" type="email"
       required aria-invalid="true">

// heard: “Email address, edit text,
//         required, invalid”
```

SCREEN READERS

Browse mode vs. Forms mode

The #1 source of “it works for me but not the screen reader” confusion. NVDA/JAWS silently switch between two modes.

Browse / Read mode

Default for reading. Single keys navigate: H = next heading, K = link, T = table, D = landmark. Keystrokes go to the screen reader, not the page.

E.g. pressing ‘b’ jumps to the next button — it does NOT type ‘b’.

Forms / Focus mode

Auto-engages when focus enters an input or custom widget. Keys pass through to the control so you can type and use arrows.

If a custom widget doesn't trigger forms mode, users can't type — a real bug.

QA cue: if Tab reaches a control but typing/arrows do nothing, forms mode isn't engaging — check the role.

SCREEN READERS

Live demo — NVDA cheat sheet

Free from nvaccess.org. NVDA key = Insert (or Caps Lock). We'll run these live on a real page.

START / STOP

NVDA + Q	Quit NVDA
Ctrl	Stop speech
NVDA + ↓	Read from here

NAVIGATE BY ELEMENT

H / Shift+H	Next / prev heading
1–6	Heading level
K	Next link
F	Next form field
D	Next landmark
T	Next table

LISTS & INSPECT

NVDA + F7	Elements list
B	Next button
Tab	Next focusable
NVDA + Tab	Read focused item

TRY IT · 3 MIN

Close your eyes — now find checkout

Put on headphones, turn the screen off, and try to add an item to the cart using only NVDA.

- Did every button say what it does?
- Could you tell when something was selected?
- Did errors announce themselves — or just turn red silently?
- How many seconds before you wanted to give up?

03

THE STANDARD

WCAG 2.2

One testable yardstick. Stop arguing about opinions — start citing success criteria.

WCAG 2.2

What it is

The Web Content Accessibility Guidelines — the W3C's international standard for accessible digital content. The version laws point to.

WCAG 2.0

2008

Foundation

WCAG 2.1

2018

+ mobile, low-vision, cognitive

WCAG 2.2

2023

+ focus, dragging, auth, help

WCAG 2.2 is backwards-compatible: meeting 2.2 means you also meet 2.1 and 2.0. It adds 9 new criteria and removes one (4.1.1 Parsing).

WCAG 2.2

How it's structured

4 PRINCIPLES — “POUR”

Perceivable

Operable

Understandable

Robust

NESTED LAYERS

4

Principles

POUR

13

Guidelines

Broad goals

86

Success Criteria

Testable pass/fail

CONFORMANCE LEVELS

LEVEL A — Must — critical blockers

LEVEL AA — The legal target

LEVEL AAA — Gold — not page-wide

AA conformance = every Level A criterion + every Level AA criterion. That's the 55 we'll walk through.

WCAG 2.2 LEVEL AA

Your 55-criterion map

AA conformance requires all 30 Level A + all 25 Level AA criteria. Here's the terrain before we go criterion by criterion.

20

criteria

Perceivable

Text alts, media, structure, color, contrast, reflow

20

criteria

Operable

Keyboard, timing, seizures, navigation, pointer & targets

13

criteria

Understandable

Language, predictability, errors, help, auth

2

criteria

Robust

Name/Role/Value, status messages

PRINCIPLE 1

Perceivable

Can people take the content in — by sight, sound, or touch?

20 CRITERIA · A + AA

1.1.1 Non-text Content

LEVEL A

Every image, icon, chart, or control that conveys meaning needs a text alternative that serves the same purpose.

COMMON FAILURES

- An icon-only “delete” button with no aria-label — a screen reader announces just “button”.
- A meaningful photo shipped with alt="" or alt="IMG_2841.jpg" instead of a real description.
- A CAPTCHA image with no text purpose and no alternative form for screen-reader users.

EXCEPTION

- Decorative or invisible images may use empty alt (alt="").
- CAPTCHAs and pre-labeled controls need only a stated purpose.

RECOMMENDED TECHNIQUES

- H37 — write a real alt attribute on every <img>
- ARIA6 / ARIA10 — aria-label or aria-labelledby on icon-only controls
- G94 / G196 — short and long text alternatives for informational images

Official: https://www.w3.org/WAI/WCAG22/Understanding/non-text-content

PERCEIVABLE · GUIDELINE 1.1 · TEXT ALTERNATIVES · EXAMPLE

1.1.1 Non-text Content

LEVEL A

THE SCENARIO

A QA engineer tests a checkout page and finds a row of icon-only buttons — trash, edit, favorite — next to each cart item. Visually clear. On NVDA, all three announce as “button” with no way to tell them apart.

✕ BEFORE — FAILS 1.1.1

```
<button class="icon-btn"
        onclick="deleteItem()">
  <svg class="trash-icon"></svg>
</button>

// Screen reader hears:
// "'button' — nothing else"
```

✓ AFTER — PASSES 1.1.1

```
<button class="icon-btn"
        aria-label="Delete item"
        onclick="deleteItem()">
  <svg aria-hidden="true"></svg>
</button>

// Screen reader hears:
// "'Delete item, button'"
```

Official: https://www.w3.org/WAI/WCAG22/Understanding/non-text-content

1.2.1 Audio-only & Video-only (Prerecorded)

LEVEL A

Prerecorded audio-only needs a transcript; prerecorded video-only needs a transcript or audio track.

COMMON FAILURES

- A podcast episode published on the site with no transcript anywhere on the page.
- A silent product-tour GIF treated as decorative when it's actually video-only content.

EXCEPTION

- Live audio/video is covered by other criteria — this applies to prerecorded only.
- Media that is itself an alternative for existing text is exempt.

RECOMMENDED TECHNIQUES

- G158 — provide a full text transcript for prerecorded audio
- G159 / G166 — audio track or transcript for video-only content

Official: https://www.w3.org/WAI/WCAG22/Understanding/audio-only-and-video-only-prerecorded

PERCEIVABLE · GUIDELINE 1.2 · TIME-BASED MEDIA · EXAMPLE

1.2.1 Audio-only & Video-only (Prerecorded)

LEVEL A

THE SCENARIO

A support page embeds a 3-minute audio walkthrough of the refund policy. It's the only place this information lives, and there's no transcript link anywhere near the player.

✕ BEFORE — FAILS 1.2.1

```
<audio controls src="refund.mp3">
</audio>
// No transcript, no text link

// Screen reader hears:
// "Just 'audio, play button' — content is unreachable"
```

✓ AFTER — PASSES 1.2.1

```
<audio controls src="refund.mp3">
</audio>
<a href="/refund-transcript">
  Read full transcript</a>

// Screen reader hears:
// "'Audio, play button' then a reachable transcript link"
```

Official: https://www.w3.org/WAI/WCAG22/Understanding/audio-only-and-video-only-prerecorded

1.2.2 Captions (Prerecorded)

LEVEL A

All prerecorded video with audio must have synchronized captions.

COMMON FAILURES

- A product demo video with spoken narration but no captions at all.
- Captions that exist but drift out of sync, or drop key spoken numbers/names.

EXCEPTION

- Media that is a labeled alternative for text is exempt.

RECOMMENDED TECHNIQUES

- H95 — use HTML track elements to add captions to <video>
- G87 / G93 — provide and sync closed captions for prerecorded video

Official: https://www.w3.org/WAI/WCAG22/Understanding/captions-prerecorded

1.2.2 Captions (Prerecorded)

LEVEL A

THE SCENARIO

A marketing team ships an autoplaying homepage video with a voiceover explaining the product. QA mutes their speakers to test and realizes there is no way to read what's being said.

✕ BEFORE — FAILS 1.2.2

```
<video src="demo.mp4" controls>
</video>
// No <track> element

// Screen reader hears:
// "Audio plays; deaf users get nothing"
```

✓ AFTER — PASSES 1.2.2

```
<video src="demo.mp4" controls>
  <track kind="captions"
    src="demo.vtt" srclang="en">
</video>

// Screen reader hears:
// "Synchronized captions render over the video"
```

Official: https://www.w3.org/WAI/WCAG22/Understanding/captions-prerecorded

1.2.3 Audio Description or Media Alternative

LEVEL A

Prerecorded video needs either audio description of important visuals or a full text alternative.

COMMON FAILURES

- A tutorial that shows steps silently on screen — a blind user hears nothing about what's happening.
- A video with narrated audio that never mentions the on-screen chart it's describing.

EXCEPTION

- If all visual information is already spoken in the audio track, no extra description is needed.

RECOMMENDED TECHNIQUES

- G78 — provide a second, audio-described version of the video
- G173 — use standard audio description conventions for timing

Official: https://www.w3.org/WAI/WCAG22/Understanding/audio-description-or-media-alternative-prerecorded

PERCEIVABLE · GUIDELINE 1.2 · TIME-BASED MEDIA · EXAMPLE

1.2.3 Audio Description or Media Alternative

LEVEL A

THE SCENARIO

An onboarding video shows a cursor clicking through settings with light background music but no narration. A blind tester can hear the music but has no idea what's being configured.

✕ BEFORE — FAILS 1.2.3

```
<video src="setup.mp4" controls>
</video>
// Visual-only steps, no narration

// Screen reader hears:
// "Background music only — steps are invisible"
```

✓ AFTER — PASSES 1.2.3

```
<video src="setup-ad.mp4" controls>
// Adds narrated description of
// each on-screen action
</video>

// Screen reader hears:
// "'Click Settings, then Privacy' is narrated aloud"
```

Official: https://www.w3.org/WAI/WCAG22/Understanding/audio-description-or-media-alternative-prerecorded

1.2.4 Captions (Live)

LEVEL AA

Live audio content in synchronized media must have real-time captions.

COMMON FAILURES

- A live-streamed webinar or town hall with no live captioning turned on.
- Live captions enabled but running many seconds behind the speaker, losing context.

EXCEPTION

- None at AA for live synchronized media with audio.

RECOMMENDED TECHNIQUES

- G9 — provide real-time captioning through a live captioning service
- G93 — caption synchronized media in real time

Official: https://www.w3.org/WAI/WCAG22/Understanding/captions-live

1.2.4 Captions (Live)

LEVEL AA

THE SCENARIO

A company-wide town hall streams live with the CEO speaking. A deaf employee joins to watch but the captioning toggle does nothing — it was never wired up for this stream.

✕ BEFORE — FAILS 1.2.4

```
<div id="live-player">
// Stream has no caption track
</div>

// Screen reader hears:
// "Speech plays; no caption track exists"
```

✓ AFTER — PASSES 1.2.4

```
<div id="live-player">
  <track kind="captions"
    src="live-cc.vtt" default>
</div>

// Screen reader hears:
// "Real-time captions scroll under the stream"
```

Official: https://www.w3.org/WAI/WCAG22/Understanding/captions-live

1.2.5 Audio Description (Prerecorded)

LEVEL AA

Prerecorded video must have audio description (at AA the text-alternative loophole of 1.2.3 is closed).

COMMON FAILURES

- A training video where on-screen diagrams are never described in the audio or a separate AD track.
- An “audio described” version that was recorded but never linked from the actual page.

EXCEPTION

- None at AA — required whenever there is meaningful visual content in prerecorded video.

RECOMMENDED TECHNIQUES

- G78 — provide audio description for prerecorded video content
- G8 — include audio description in the soundtrack itself

Official: https://www.w3.org/WAI/WCAG22/Understanding/audio-description-prerecorded

PERCEIVABLE · GUIDELINE 1.2 · TIME-BASED MEDIA · EXAMPLE

1.2.5 Audio Description (Prerecorded)

LEVEL AA

THE SCENARIO

A quarterly results video cuts between a speaker and full-screen charts with no verbal explanation of the numbers. Sighted viewers read the chart; blind viewers hear silence during those cuts.

✕ BEFORE — FAILS 1.2.5

```
<video src="q3-results.mp4">
</video>
// Chart cuts have no narration

// Screen reader hears:
// "Silence during every chart cutaway"
```

✓ AFTER — PASSES 1.2.5

```
<video src="q3-results-ad.mp4">
// Adds audio description track
// during every silent chart cut
</video>

// Screen reader hears:
// "'Revenue chart shows 12% growth' is narrated"
```

Official: https://www.w3.org/WAI/WCAG22/Understanding/audio-description-prerecorded

PERCEIVABLE · GUIDELINE 1.3 · ADAPTABLE

1.3.1 Info and Relationships

LEVEL A

Structure conveyed visually (headings, lists, tables, labels) must also be in the markup/semantics.

COMMON FAILURES

- Bold, large text used as a “heading” with no real <h2> behind it.
- A data table built entirely from styled s instead of <table>/<th>/<td>.
- Required fields shown only by a red asterisk with no programmatic association.

EXCEPTION

- None — structure conveyed visually must also exist in markup.

RECOMMENDED TECHNIQUES

- H42 / H43 — use real heading and table markup, not styled divs
- H44 / H65 — associate every label with its input programmatically

Official: https://www.w3.org/WAI/WCAG22/Understanding/info-and-relationships

PERCEIVABLE · GUIDELINE 1.3 · ADAPTABLE · EXAMPLE

1.3.1 Info and Relationships

LEVEL A

THE SCENARIO

A pricing page has three “column headers” styled with bold 20px text but built from s. A screen reader user asks NVDA to jump by heading and finds nothing — there's no structure to navigate.

✕ BEFORE — FAILS 1.3.1

```
<div class="col-header">
  Enterprise Plan</div>
// Looks like a heading, isn't one

// Screen reader hears:
// "Reads as plain text; 'next heading' skips it"
```

✓ AFTER — PASSES 1.3.1

```
<h2 class="col-header">
  Enterprise Plan</h2>

// Screen reader hears:
// "'Enterprise Plan, heading level 2'"
```

Official: https://www.w3.org/WAI/WCAG22/Understanding/info-and-relationships

PERCEIVABLE · GUIDELINE 1.3 · ADAPTABLE

1.3.2 Meaningful Sequence

LEVEL A

When reading order matters, the DOM order must match the meaningful order.

COMMON FAILURES

- CSS positions a sidebar visually after the article, but it reads first in the DOM, scrambling the story.
- A CSS grid reorders cards visually while the underlying markup order stays unrelated to reading order.

EXCEPTION

- If the sequence doesn't affect meaning, any order is acceptable.

RECOMMENDED TECHNIQUES

- G57 — order content in the DOM to match its meaningful reading order
- C27 — test that CSS positioning doesn't create a mismatched sequence

Official: https://www.w3.org/WAI/WCAG22/Understanding/meaningful-sequence

PERCEIVABLE · GUIDELINE 1.3 · ADAPTABLE · EXAMPLE

1.3.2 Meaningful Sequence

LEVEL A

THE SCENARIO

A blog layout uses CSS order to show a pull-quote before the intro paragraph visually, but the DOM has the pull-quote at the very end of the article. Reading it with CSS off makes no sense.

✕ BEFORE — FAILS 1.3.2

```
<style>.pull-quote{order:-1}
</style>
// DOM order != reading order

// Screen reader hears:
// "Pull-quote reads last, after the conclusion"
```

✓ AFTER — PASSES 1.3.2

```
// Move markup to match
// the intended reading order
<p class="pull-quote">...</p>
<p>Intro paragraph...</p>

// Screen reader hears:
// "Pull-quote reads in its intended place"
```

Official: https://www.w3.org/WAI/WCAG22/Understanding/meaningful-sequence

PERCEIVABLE · GUIDELINE 1.3 · ADAPTABLE

1.3.3 Sensory Characteristics

LEVEL A

Instructions must not rely solely on shape, size, position, orientation, or sound.

COMMON FAILURES

- “Click the round green button on the right” — meaningless to a screen-reader or color-blind user.
- “See the box below” in a layout where reading order and visual order don't match.

EXCEPTION

- None — instructions must not rely solely on shape, position, or sound.

RECOMMENDED TECHNIQUES

- G96 — pair sensory cues (shape, position) with a text label
- F26 — avoid using only a graphical symbol to identify content

Official: https://www.w3.org/WAI/WCAG22/Understanding/sensory-characteristics

PERCEIVABLE · GUIDELINE 1.3 · ADAPTABLE · EXAMPLE

1.3.3 Sensory Characteristics

LEVEL A

THE SCENARIO

A checkout guide tells users to “click the blue button on the right to continue.” A color-blind tester can see a button but can't confirm it's the right one from color alone, and a screen-reader user has no “right side” to go on.

✕ BEFORE — FAILS 1.3.3

```
<p>Click the round green
   button on the right.</p>

// Screen reader hears:
// "'Round green button on the right' — unusable"
```

✓ AFTER — PASSES 1.3.3

```
<p>Click "Continue"
   (the green button).</p>

// Screen reader hears:
// "'Click Continue' — identifiable by name"
```

Official: https://www.w3.org/WAI/WCAG22/Understanding/sensory-characteristics

1.3.4 Orientation

LEVEL AA

Content must not lock to a single orientation (portrait or landscape) unless essential.

COMMON FAILURES

- A mobile web app that forces landscape and shows “rotate your device” in portrait.
- A dashboard that only renders its charts correctly in landscape, blocking portrait entirely.

EXCEPTION

- When a specific orientation is essential (e.g. a piano app, a bank check-capture screen).

RECOMMENDED TECHNIQUES

- F97 — avoid restricting display and operation to a single orientation
- SCR30 — unlock orientation with the CSS/JS screen orientation API

Official: https://www.w3.org/WAI/WCAG22/Understanding/orientation

1.3.4 Orientation

LEVEL AA

THE SCENARIO

A wheelchair-mounted-device user opens a reporting dashboard on a tablet fixed in portrait mode. The page detects portrait and shows a full-screen “please rotate” overlay that can't be dismissed.

✕ BEFORE — FAILS 1.3.4

```
<meta name="viewport"
  content="orientation=landscape">

// Screen reader hears:
// "Content is blocked behind a rotate prompt"
```

✓ AFTER — PASSES 1.3.4

```
// Remove the forced-orientation
// lock; use responsive CSS
// for both orientations instead

// Screen reader hears:
// "Dashboard renders and works in either orientation"
```

Official: https://www.w3.org/WAI/WCAG22/Understanding/orientation

PERCEIVABLE · GUIDELINE 1.3 · ADAPTABLE

1.3.5 Identify Input Purpose

LEVEL AA

Inputs collecting user info must expose their purpose programmatically (HTML autocomplete tokens).

COMMON FAILURES

- A name/email/phone form with no autocomplete attributes, so autofill and personalization fail.
- An autocomplete value set to the wrong token, silently breaking browser autofill.

EXCEPTION

- Only applies to fields about the user, where a matching defined token exists.

RECOMMENDED TECHNIQUES

- H98 — use the correct HTML autocomplete token for each user-info field
- ARIA1 — use aria-label alongside autocomplete when the visible label is ambiguous

Official: https://www.w3.org/WAI/WCAG22/Understanding/identify-input-purpose

PERCEIVABLE · GUIDELINE 1.3 · ADAPTABLE · EXAMPLE

1.3.5 Identify Input Purpose

LEVEL AA

THE SCENARIO

A checkout form has separate first-name and last-name fields with no autocomplete tokens. Testers with password managers and browser autofill find every field blank on page load.

✕ BEFORE — FAILS 1.3.5

```
<input type="text" id="fname"
  name="fname">
// No autocomplete token

// Screen reader hears:
// "Autofill has no idea what this field is for"
```

✓ AFTER — PASSES 1.3.5

```
<input type="text" id="fname"
  name="fname"
  autocomplete="given-name">

// Screen reader hears:
// "Browser correctly offers the saved first name"
```

Official: https://www.w3.org/WAI/WCAG22/Understanding/identify-input-purpose

1.4.1 Use of Color

LEVEL A

Color must not be the only visual means of conveying info, indicating action, or distinguishing elements.

COMMON FAILURES

- Required fields shown only in red text, with no icon, label, or symbol.
- Links distinguished from body text only by color, with no underline or weight change.

EXCEPTION

- None — color must never be the sole visual cue.

RECOMMENDED TECHNIQUES

- G14 — ensure information isn't conveyed by color alone
- G182 — pair color-coding with an additional visual cue (icon, pattern, text)

Official: https://www.w3.org/WAI/WCAG22/Understanding/use-of-color

1.4.1 Use of Color

LEVEL A

THE SCENARIO

A form marks required fields in red but adds no asterisk or text label. A color-blind QA tester can't tell which fields are mandatory until they submit and see an error.

✕ BEFORE — FAILS 1.4.1

```
<label style="color:red">
  Email</label>

// Screen reader hears:
// "'Email' — no indication it's required"
```

✓ AFTER — PASSES 1.4.1

```
<label>
  Email <span aria-hidden="true">*</span>
  <span class="sr-only">required</span>
</label>

// Screen reader hears:
// "'Email, required' is announced"
```

Official: https://www.w3.org/WAI/WCAG22/Understanding/use-of-color

1.4.2 Audio Control

LEVEL A

Audio that plays automatically for >3s must have a way to pause, stop, or control volume independently.

COMMON FAILURES

- A homepage video auto-plays sound with no visible pause control anywhere on the page.
- A pause control that exists but is buried below the fold or hidden until hover.

EXCEPTION

- Audio that plays for less than 3 seconds is exempt.

RECOMMENDED TECHNIQUES

- G60 — provide a mechanism to stop, pause, or mute automatic audio
- G170 — provide the pause control as the first focusable element

Official: https://www.w3.org/WAI/WCAG22/Understanding/audio-control

1.4.2 Audio Control

LEVEL A

THE SCENARIO

A landing page autoplays a background video with music. A screen-reader user tabbing through the page has no way to silence it while trying to hear their own screen reader over it.

✕ BEFORE — FAILS 1.4.2

```
<video autoplay src="bg.mp4">
</video>
// No pause/mute control on page

// Screen reader hears:
// "Music plays over the screen reader with no way to stop it"
```

✓ AFTER — PASSES 1.4.2

```
<video autoplay src="bg.mp4">
</video>
<button aria-label="Pause background
  audio">⏸ </button>

// Screen reader hears:
// "'Pause background audio, button' is reachable first"
```

Official: https://www.w3.org/WAI/WCAG22/Understanding/audio-control

PERCEIVABLE · GUIDELINE 1.4 · DISTINGUISHABLE

1.4.3 Contrast (Minimum)

LEVEL AA

Text contrast ≥4.5:1 (normal) or ≥3:1 (large: ≥18.66px bold / ≥24px).

COMMON FAILURES

- Light gray placeholder text sitting around 2.8:1 against a white background.
- White text on a light-teal button that measures well under 4.5:1.

EXCEPTION

- Disabled controls, logos, and purely decorative or incidental text are exempt.

RECOMMENDED TECHNIQUES

- G18 — ensure text contrast is at least 4.5:1 against its background
- G145 — use 3:1 for large-scale text (18.66px bold / 24px regular)

Official: https://www.w3.org/WAI/WCAG22/Understanding/contrast-minimum

1.4.3 Contrast (Minimum)

LEVEL AA

THE SCENARIO

A design system ships a “subtle” gray for helper text under form fields. A contrast check shows 2.9:1 — comfortably readable for the designer, nearly invisible for a user with low vision.

✕ BEFORE — FAILS 1.4.3

```
<p style="color:#999999;
  background:#ffffff">
  We'll never share your email</p>

// Screen reader hears:
// "N/A – visual defect, not announced"
```

✓ AFTER — PASSES 1.4.3

```
<p style="color:#595959;
  background:#ffffff">
  We'll never share your email</p>

// Screen reader hears:
// "Text now measures 7:1 contrast"
```

Official: https://www.w3.org/WAI/WCAG22/Understanding/contrast-minimum

1.4.4 Resize Text

LEVEL AA

Text must scale up to 200% without loss of content or function (no horizontal scroll trap, no clipping).

COMMON FAILURES

- At 200% browser zoom a fixed-width menu overlaps content and the “Buy” button is cut off.
- A fixed-height card clips its own text once the font scales up.

EXCEPTION

- Captions and images of text are exempt.

RECOMMENDED TECHNIQUES

- G142 — use rem/em units so text scales with browser zoom
- C28 — avoid fixed pixel heights that clip scaled text

Official: https://www.w3.org/WAI/WCAG22/Understanding/resize-text

1.4.4 Resize Text

LEVEL AA

THE SCENARIO

A tester zooms a pricing page to 200% to check readability for low-vision users. The price card has a fixed height in pixels, so the price and “Buy now” button get clipped off the bottom of the card.

✕ BEFORE — FAILS 1.4.4

```
<div style="height:120px;
  overflow:hidden">
  <p>$29/month</p>
</div>

// Screen reader hears:
// "N/A — visual clipping, not announced"
```

✓ AFTER — PASSES 1.4.4

```
<div style="min-height:120px">
  <p>$29/month</p>
</div>

// Screen reader hears:
// "Card grows to fit the scaled text, nothing clipped"
```

Official: https://www.w3.org/WAI/WCAG22/Understanding/resize-text

1.4.5 Images of Text

LEVEL AA

A

Use real text instead of images of text when the same presentation is possible.

COMMON FAILURES

- A promo banner where the headline and body copy are baked into a JPG image.
- A screenshot of a code snippet used instead of real, selectable text.

EXCEPTION

- Logos/logotypes, and cases where a specific visual presentation is essential.

RECOMMENDED TECHNIQUES

- C22 — use real styled text (CSS) instead of an image of text
- G140 — separate information and structure from presentation to allow real text

Official: https://www.w3.org/WAI/WCAG22/Understanding/images-of-text

1.4.5 Images of Text

LEVEL AA

A

THE SCENARIO

A marketing banner ships as a single JPG with the headline “Save 30% Today” baked in. Zooming in blurs it, and it can't be read aloud, translated, or resized.

✕ BEFORE — FAILS 1.4.5

```
<img src="banner.jpg"
  alt="Save 30% Today">

// Screen reader hears:
// "Alt text is read once, but the text can't resize or reflow"
```

✓ AFTER — PASSES 1.4.5

```
<div class="banner">
  <h2>Save 30% Today</h2>
</div>

// Screen reader hears:
// "Real, resizable, selectable heading text"
```

Official: https://www.w3.org/WAI/WCAG22/Understanding/images-of-text

1.4.10 Reflow

LEVEL AA

Content reflows to a single column at 320 CSS px wide (400% zoom) with no 2-D scrolling.

COMMON FAILURES

- At 320px width the page still requires horizontal scrolling to read a single line.
- A sticky header eats so much vertical space at narrow widths that content becomes unusable.

EXCEPTION

- Content requiring 2-D layout is exempt: data tables, maps, complex images, toolbars, video.

RECOMMENDED TECHNIQUES

- C32 / C31 — use CSS media queries and relative units for a single-column reflow
- G206 — use CSS media queries to reflow content at narrow viewports

Official: https://www.w3.org/WAI/WCAG22/Understanding/reflow

1.4.10 Reflow

LEVEL AA

THE SCENARIO

A tester sets the browser to 400% zoom (equivalent to a 320px viewport) to check reflow. The page's fixed-width table forces horizontal scrolling just to read one row of text.

✕ BEFORE — FAILS 1.4.10

```
<div style="width:1200px">
  ...page content...
</div>

// Screen reader hears:
// "N/A — requires 2-D scrolling to read"
```

✓ AFTER — PASSES 1.4.10

```
<div style="max-width:1200px;
  width:100%">
  ...page content...
</div>

// Screen reader hears:
// "Content reflows to one column, no 2-D scroll"
```

Official: https://www.w3.org/WAI/WCAG22/Understanding/reflow

1.4.11 Non-text Contrast

LEVEL AA

UI components and meaningful graphics need ≥3:1 contrast against adjacent colors.

COMMON FAILURES

- A 1.5:1 gray input border that's barely visible against the page background.
- An icon-only button whose glyph barely shows against its own background fill.

EXCEPTION

- Inactive/disabled components, and graphics where a specific presentation is essential.

RECOMMENDED TECHNIQUES

- G195 — ensure a state or component has enough contrast against its background
- G174 — make focus/state indicators visible enough to meet 3:1 contrast

Official: https://www.w3.org/WAI/WCAG22/Understanding/non-text-contrast

1.4.11 Non-text Contrast

LEVEL AA

THE SCENARIO

A form's input borders are styled a very light gray to look “clean.” A low-vision tester can't tell where one field ends and the next begins without clicking around blindly.

✕ BEFORE — FAILS 1.4.11

```
<input style="border:
  1px solid #e0e0e0">

// Screen reader hears:
// "N/A — visual defect, not announced"
```

✓ AFTER — PASSES 1.4.11

```
<input style="border:
  1px solid #767676">

// Screen reader hears:
// "Border now meets 3:1 against the page background"
```

Official: https://www.w3.org/WAI/WCAG22/Understanding/non-text-contrast

1.4.12 Text Spacing

LEVEL AA

No loss of content when users override line-height to 1.5×, paragraph 2×, letter 0.12×, word 0.16×.

COMMON FAILURES

- Applying the Text Spacing bookmarklet causes text to overlap or clip inside a fixed-height button.
- Line-height overrides make multi-line labels overlap the row below them.

EXCEPTION

- Languages/scripts that don't use these spacing properties are exempt.

RECOMMENDED TECHNIQUES

- C36 / C35 — use relative line-height and spacing values instead of fixed pixels
- G188 — avoid fixed-height containers that clip when spacing is overridden

Official: https://www.w3.org/WAI/WCAG22/Understanding/text-spacing

1.4.12 Text Spacing

LEVEL AA

THE SCENARIO

A tester applies the official text-spacing bookmarklet (line-height 1.5, paragraph spacing 2x) to a card component. Fixed-height buttons start clipping their own label text.

✕ BEFORE — FAILS 1.4.12

```
<button style="height:32px;
  overflow:hidden">Submit</button>

// Screen reader hears:
// "N/A – visual clipping, not announced"
```

✓ AFTER — PASSES 1.4.12

```
<button style="min-height:32px;
  padding:8px 16px">Submit</button>

// Screen reader hears:
// "Button grows to fit user-overridden spacing"
```

Official: https://www.w3.org/WAI/WCAG22/Understanding/text-spacing

1.4.13 Content on Hover or Focus

LEVEL AA

Tooltips/popovers on hover/focus must be dismissable, hoverable, and persistent.

COMMON FAILURES

- A tooltip vanishes the instant the pointer moves toward it, so it can never be read.
- A popover with no way to dismiss it except moving focus away entirely.

EXCEPTION

- Browser-native tooltips (the title attribute) and user-agent-controlled content are exempt.

RECOMMENDED TECHNIQUES

- SCR39 — make hover/focus content dismissable, hoverable, and persistent
- F95 — avoid content that disappears before it can be interacted with

Official: https://www.w3.org/WAI/WCAG22/Understanding/content-on-hover-or-focus

1.4.13 Content on Hover or Focus

LEVEL AA

THE SCENARIO

A pricing table shows extra plan details in a tooltip on hover. As soon as a tester moves the mouse toward the tooltip to read it fully, it disappears — the hover target was too small.

✕ BEFORE — FAILS 1.4.13

```
<span onmouseleave=
  "hideTooltip()">Info</span>

// Screen reader hears:
// "Tooltip closes before it can be read or hovered"
```

✓ AFTER — PASSES 1.4.13

```
<span onmouseenter=
  "showTooltip()">Info</span>
// dismiss only via Esc or
// moving away from tooltip itself

// Screen reader hears:
// "Tooltip stays open while hovered; Esc dismisses it"
```

Official: https://www.w3.org/WAI/WCAG22/Understanding/content-on-hover-or-focus

SPOT THE BUG · 30 SEC

Use of Color (1.4.1)

```
<a href="/terms" style="color:#1a73e8">
  Read the terms
</a>

/* link looks identical to body
   text except for its colour */
```

Why might a colour-blind user miss this link?

Answer

Color is the only cue. Add an underline, or ensure 3:1 contrast vs. body text plus a non-color cue on hover/focus.

PRINCIPLE 2

Operable

Can people drive the interface — by keyboard, touch, voice, or switch?

20 CRITERIA · A + AA

2.1.1 Keyboard

LEVEL A

All functionality must be operable with a keyboard alone.

COMMON FAILURES

- A custom dropdown opens on mouse click but cannot be opened or navigated via keyboard.
- A hover-only mega-menu that never appears for keyboard-only users.

EXCEPTION

- Functions that inherently require path-dependent input, e.g. freehand drawing.

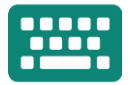
RECOMMENDED TECHNIQUES

- G202 — ensure all interactive functions are operable through the keyboard
- SCR35 / SCR29 — add keydown handlers to custom widgets so they respond to Enter/Space

Official: https://www.w3.org/WAI/WCAG22/Understanding/keyboard

2.1.1 Keyboard

LEVEL A



THE SCENARIO

A QA engineer unplugs the mouse and tries to open the account-settings dropdown built from styled s. Tab reaches it, but nothing happens on Enter, Space, or arrow keys.

✕ BEFORE — FAILS 2.1.1

```
<div class="dropdown"
  onclick="toggle()">Menu</div>

// Screen reader hears:
// "Focus lands on it, but no key opens it"
```

✓ AFTER — PASSES 2.1.1

```
<button class="dropdown"
  aria-expanded="false"
  onclick="toggle()">Menu</button>

// Screen reader hears:
// "Enter, Space, or click all open the menu"
```

Official: https://www.w3.org/WAI/WCAG22/Understanding/keyboard

2.1.2 No Keyboard Trap

LEVEL A

Keyboard focus must be able to move away from any component using standard keys.

COMMON FAILURES

- A modal or embedded widget traps Tab so focus can never leave it.
- A third-party chat widget that swallows Tab and Shift+Tab entirely.

EXCEPTION

- None — if non-standard exit keys are needed, users must be told how to use them.

RECOMMENDED TECHNIQUES

- G21 — ensure users can navigate away from any component using the keyboard
- F10 — avoid trapping focus without a documented exit key

Official: https://www.w3.org/WAI/WCAG22/Understanding/no-keyboard-trap

2.1.2 No Keyboard Trap

LEVEL A

THE SCENARIO

A QA engineer opens a video-embed widget and tabs into it. Tab and Shift+Tab both stay locked inside the iframe forever — there is no way back to the rest of the page.

✕ BEFORE — FAILS 2.1.2

```
<div class="modal" tabindex="-1">
// Tab never exits this container
</div>

// Screen reader hears:
// "Tab loops inside forever, page is unreachable"
```

✓ AFTER — PASSES 2.1.2

```
<div class="modal" role="dialog"
  aria-modal="true">
// Tab cycles within, Esc exits
</div>

// Screen reader hears:
// "Esc (or completing the flow) returns focus to the page"
```

Official: https://www.w3.org/WAI/WCAG22/Understanding/no-keyboard-trap

2.1.4 Character Key Shortcuts

LEVEL A

Single-character shortcuts must be turn-off-able, remappable, or active only on focus.

COMMON FAILURES

- Pressing “r” anywhere on the page triggers “reply” — dictation users fire it by accident.
- A single-letter shortcut with no setting anywhere to turn it off or remap it.

EXCEPTION

- Shortcuts that require a non-printing key (Ctrl, Alt) plus a character are exempt.

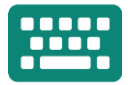
RECOMMENDED TECHNIQUES

- G217 — make single-key shortcuts remappable or turn-off-able
- SCR20 — activate a shortcut only when a specific component has focus

Official: https://www.w3.org/WAI/WCAG22/Understanding/character-key-shortcuts

2.1.4 Character Key Shortcuts

LEVEL A



THE SCENARIO

A speech-input user dictating an email accidentally says a word starting with “r”, which the page interprets as the “reply” shortcut and opens an unwanted reply box.

✕ BEFORE — FAILS 2.1.4

```
document.addEventListener(
  'keydown', e => {
  if (e.key === 'r') reply();
});

// Screen reader hears:
// "Speech input accidentally triggers 'reply'"
```

✓ AFTER — PASSES 2.1.4

```
document.addEventListener(
  'keydown', e => {
  if (e.key === 'r' &&
      shortcutsEnabled) reply();
});

// Screen reader hears:
// "Shortcut only fires when explicitly enabled"
```

Official: https://www.w3.org/WAI/WCAG22/Understanding/character-key-shortcuts

2.2.1 Timing Adjustable

LEVEL A

Users must be able to turn off, adjust, or extend time limits.

COMMON FAILURES

- A form session expires in 60 seconds with no warning and no way to extend it.
- A countdown timer that resets the whole form instead of just warning the user.

EXCEPTION

- Real-time events (auctions), essential limits, and limits over 20 hours are exempt.

RECOMMENDED TECHNIQUES

- G133 — provide a warning and let users extend the time limit with a simple action
- SCR16 — allow users to turn off or adjust a time limit before it expires

Official: https://www.w3.org/WAI/WCAG22/Understanding/timing-adjustable

2.2.1 Timing Adjustable

LEVEL A

THE SCENARIO

A user with a motor disability fills out a long insurance form slowly using switch access. The session silently expires at 60 seconds and wipes everything they'd entered, with no warning.

✕ BEFORE — FAILS 2.2.1

```
setTimeout(() => logout(),
  60000);
// No warning before logout

// Screen reader hears:
// "Session ends with no announcement"
```

✓ AFTER — PASSES 2.2.1

```
setTimeout(() => showWarning(
  {extendable: true}), 40000);
// Warns at 40s, offers extension

// Screen reader hears:
// "'Session expiring in 20s, press Extend' is announced"
```

Official: https://www.w3.org/WAI/WCAG22/Understanding/timing-adjustable

2.2.2 Pause, Stop, Hide

LEVEL A

Moving/blinking/auto-updating content lasting >5s must be pausable, stoppable, or hideable.

COMMON FAILURES

- An auto-rotating carousel with no pause control keeps stealing focus and attention.
- An animated loading spinner that never stops, even after the page has fully loaded.

EXCEPTION

- Movement essential to the activity is exempt, e.g. a progress bar or a game.

RECOMMENDED TECHNIQUES

- G152 — add a visible pause/stop control to auto-updating content
- G187 — stop blinking text after a fixed number of cycles

Official: https://www.w3.org/WAI/WCAG22/Understanding/pause-stop-hide

2.2.2 Pause, Stop, Hide

LEVEL A

THE SCENARIO

A homepage carousel auto-advances every 4 seconds. A user with ADHD trying to read the second slide's text keeps losing their place as it jumps to the next slide before they finish.

✕ BEFORE — FAILS 2.2.2

```
<div class="carousel"
  data-autoplay="4000">
// No pause control anywhere
</div>

// Screen reader hears:
// "Slides change mid-read with no way to stop them"
```

✓ AFTER — PASSES 2.2.2

```
<div class="carousel"
  data-autoplay="4000">
  <button aria-label="Pause
    carousel">⏸</button>
</div>

// Screen reader hears:
// "'Pause carousel, button' lets users stop the motion"
```

Official: https://www.w3.org/WAI/WCAG22/Understanding/pause-stop-hide

2.3.1 Three Flashes or Below Threshold

LEVEL A

Nothing flashes more than three times per second (or stays under the flash thresholds).

COMMON FAILURES

- A promo animation strobes rapidly, risking photosensitive seizures.
- A loading animation that flashes bright/dark faster than 3 times per second.

EXCEPTION

- Flashes below the general/red flash thresholds and small enough in screen area.

RECOMMENDED TECHNIQUES

- G19 — keep flashing below the general flash and red flash thresholds
- G176 — control flash rate and screen area for animated content

Official: https://www.w3.org/WAI/WCAG22/Understanding/three-flashes-or-below-threshold

2.3.1 Three Flashes or Below Threshold

LEVEL A

THE SCENARIO

A game-like promo widget flashes a bright yellow background rapidly to grab attention. QA runs it through PEAT and finds it strobes well above the safe threshold.

✕ BEFORE — FAILS 2.3.1

```
@keyframes flash {
  0%,100% {background:#fff}
  50% {background:#ff0}
}
// animation: flash 0.2s infinite

// Screen reader hears:
// "N/A — visual seizure risk, not announced"
```

✓ AFTER — PASSES 2.3.1

```
@keyframes flash {
  0%,100% {background:#fff}
  50% {opacity:0.9}
}
// Slower, low-contrast pulse

// Screen reader hears:
// "Pulse rate and contrast are within safe thresholds"
```

Official: https://www.w3.org/WAI/WCAG22/Understanding/three-flashes-or-below-threshold

2.4.1 Bypass Blocks

LEVEL A

Provide a way to skip repeated blocks (skip link, landmarks, or headings).

COMMON FAILURES

- No “skip to main content” link, so keyboard users Tab through 40 nav links on every page.
- A skip link that's visually hidden and never becomes visible on focus.

EXCEPTION

- None, but any one mechanism — skip link, landmarks, or headings — can satisfy it.

RECOMMENDED TECHNIQUES

- G1 — add a link at the top of the page to skip to the main content
- ARIA11 — use landmark roles to identify repeated regions of a page

Official: https://www.w3.org/WAI/WCAG22/Understanding/bypass-blocks

2.4.1 Bypass Blocks

LEVEL A

THE SCENARIO

A keyboard-only tester lands on a page with a 40-item navigation menu before the content. Every single page load means tabbing through all 40 links again just to reach the article.

✕ BEFORE — FAILS 2.4.1

```
<nav>...40 links...</nav>
<main>Article content</main>
// No way to bypass the nav

// Screen reader hears:
// "40 links must be tabbed through every page"
```

✓ AFTER — PASSES 2.4.1

```
<a class="skip-link"
  href="#main">Skip to content</a>
<nav>...40 links...</nav>
<main id="main">...</main>

// Screen reader hears:
// "'Skip to content' link appears first on Tab"
```

Official: https://www.w3.org/WAI/WCAG22/Understanding/bypass-blocks

2.4.2 Page Titled

LEVEL A

Each page has a descriptive, unique <title>.

COMMON FAILURES

- Every route in a single-page app shows “React App” in the browser tab.
- A page title that never updates when navigating between views in an SPA.

EXCEPTION

- None — every page needs a descriptive, unique title.

RECOMMENDED TECHNIQUES

- G88 — provide a descriptive title for each page
- H25 — set the page title in every route, including client-side navigation

Official: https://www.w3.org/WAI/WCAG22/Understanding/page-titled

2.4.2 Page Titled

LEVEL A

THE SCENARIO

A tester navigates a single-page app between the dashboard, settings, and billing screens. The browser tab always reads “React App,” so a screen-reader user has no idea a page changed.

✕ BEFORE — FAILS 2.4.2

```
<title>React App</title>
// Never updates on route change

// Screen reader hears:
// "'React App' is announced on every navigation"
```

✓ AFTER — PASSES 2.4.2

```
router.afterEach(to => {
  document.title =
    `${to.name} · Acme`;
});

// Screen reader hears:
// "'Billing · Acme' announces the actual page"
```

Official: https://www.w3.org/WAI/WCAG22/Understanding/page-titled

2.4.3 Focus Order

LEVEL A

Focus moves in an order that preserves meaning and operability.

COMMON FAILURES

- Opening a modal leaves keyboard focus behind it, still on the page underneath.
- Positive tabindex values scramble the Tab order away from the visual layout.

EXCEPTION

- None — focus order must preserve meaning and operability.

RECOMMENDED TECHNIQUES

- G59 — order focusable elements in a meaningful sequence
- SCR26 — move focus into a newly opened dialog when it appears

Official: https://www.w3.org/WAI/WCAG22/Understanding/focus-order

OPERABLE · GUIDELINE 2.4 · NAVIGABLE · EXAMPLE

2.4.3 Focus Order

LEVEL A

THE SCENARIO

A tester opens a “confirm delete” modal. Focus stays on the trash icon behind it, so pressing Tab moves through the rest of the hidden page instead of the modal's own buttons.

✕ BEFORE — FAILS 2.4.3

```
openModal();
// Focus never moves into the modal

// Screen reader hears:
// "Tab continues through the page behind the modal"
```

✓ AFTER — PASSES 2.4.3

```
openModal();
modal.querySelector(
  '[data-autofocus]').focus();

// Screen reader hears:
// "Focus lands on the modal's first control immediately"
```

Official: https://www.w3.org/WAI/WCAG22/Understanding/focus-order

2.4.4 Link Purpose (In Context)

LEVEL A

The purpose of each link is clear from its text or its programmatic context.

COMMON FAILURES

- A page full of “Read more” links that all sound identical out of context.
- “Click here” used as the only link text, with the destination never named.

EXCEPTION

- When the purpose would be ambiguous to users in general anyway.

RECOMMENDED TECHNIQUES

- H30 — give link text that describes its purpose without needing context
- G91 — provide a meaningful accessible name for every link

Official: https://www.w3.org/WAI/WCAG22/Understanding/link-purpose-in-context

OPERABLE · GUIDELINE 2.4 · NAVIGABLE · EXAMPLE

2.4.4 Link Purpose (In Context)

LEVEL A

THE SCENARIO

A screen-reader user pulls up NVDA's links list on a blog index page and hears “Read more, read more, read more” six times in a row — no way to tell which article is which.

✕ BEFORE — FAILS 2.4.4

```
<a href="/post/42">Read more</a>

// Screen reader hears:
// "'Read more' — repeated identically, six times"
```

✓ AFTER — PASSES 2.4.4

```
<a href="/post/42">
  Read more about accessible forms
</a>

// Screen reader hears:
// "'Read more about accessible forms'"
```

Official: https://www.w3.org/WAI/WCAG22/Understanding/link-purpose-in-context

2.4.5 Multiple Ways

LEVEL AA

More than one way to locate a page within a set (search, sitemap, nav, index).

COMMON FAILURES

- A site where the only way to reach a page is clicking through one linear flow.
- A large content site with no search, sitemap, or index anywhere.

EXCEPTION

- Pages that are a step in a process, e.g. a checkout step, are exempt.

RECOMMENDED TECHNIQUES

- G125 — provide links to related content, such as a site index
- G64 — provide a table of contents or sitemap for navigating a large site

Official: https://www.w3.org/WAI/WCAG22/Understanding/multiple-ways

OPERABLE · GUIDELINE 2.4 · NAVIGABLE · EXAMPLE

2.4.5 Multiple Ways

LEVEL AA

THE SCENARIO

A knowledge-base article can only be reached by clicking through a specific support flow. A user who lands elsewhere with a direct need has no search bar and no sitemap to find it another way.

✕ BEFORE — FAILS 2.4.5

```
<nav>
  <a href="/support">Support</a>
</nav>
// No search, no sitemap link

// Screen reader hears:
// "Only one linear path exists to any article"
```

✓ AFTER — PASSES 2.4.5

```
<nav>
  <a href="/support">Support</a>
  <a href="/sitemap">Sitemap</a>
  <search-box/>
</nav>

// Screen reader hears:
// "Search and sitemap offer two more ways in"
```

Official: https://www.w3.org/WAI/WCAG22/Understanding/multiple-ways

OPERABLE · GUIDELINE 2.4 · NAVIGABLE

2.4.6 Headings and Labels

LEVEL AA

T

Headings and labels describe their topic or purpose.

COMMON FAILURES

- Form labels that all say “Field” with no indication of what belongs there.
- Section headings that just read “Section 1”, “Section 2” with no topic named.

EXCEPTION

- None — this is about the quality of headings/labels, not just their presence.

RECOMMENDED TECHNIQUES

- G130 — give headings that describe the topic or purpose of the content
- G131 — give labels that describe the purpose of the form control

Official: https://www.w3.org/WAI/WCAG22/Understanding/headings-and-labels

OPERABLE · GUIDELINE 2.4 · NAVIGABLE · EXAMPLE

2.4.6 Headings and Labels

LEVEL AA

T

THE SCENARIO

A settings form has three text inputs all labeled “Field.” The markup technically has labels (passing 1.3.1), but a screen-reader user has no idea which field is name, email, or phone.

✕ BEFORE — FAILS 2.4.6

```
<label for="f1">Field</label>
<input id="f1">

// Screen reader hears:
// "'Field, edit text' — no idea what to enter"
```

✓ AFTER — PASSES 2.4.6

```
<label for="f1">
  Phone number</label>
<input id="f1">

// Screen reader hears:
// "'Phone number, edit text'"
```

Official: https://www.w3.org/WAI/WCAG22/Understanding/headings-and-labels

2.4.7 Focus Visible

LEVEL AA

Keyboard focus must have a visible indicator.

COMMON FAILURES

- A global outline:none in the CSS reset removes all focus rings site-wide.
- A custom focus ring that's only 1px and barely visible against the background.

EXCEPTION

- None — keyboard focus must always have a visible indicator.

RECOMMENDED TECHNIQUES

- G149 — use a visible, high-contrast style change to indicate keyboard focus
- G165 — use a focus style that meets 3:1 contrast against adjacent colors

Official: https://www.w3.org/WAI/WCAG22/Understanding/focus-visible

2.4.7 Focus Visible

LEVEL AA

THE SCENARIO

A design system's CSS reset includes *{outline:none} to “clean up” the look. A keyboard-only tester tabs through the nav and has no idea which link is currently focused.

✕ BEFORE — FAILS 2.4.7

```
<style>*{outline:none}</style>

// Screen reader hears:
// "N/A — visual defect, but keyboard users are lost"
```

✓ AFTER — PASSES 2.4.7

```
<style>:focus-visible{
  outline:3px solid #1a73e8;
  outline-offset:2px
}</style>

// Screen reader hears:
// "A clear 3px ring shows exactly what has focus"
```

Official: https://www.w3.org/WAI/WCAG22/Understanding/focus-visible

2.4.11 Focus Not Obscured (Minimum)

LEVEL AA

When an element gets focus, it is not entirely hidden by other content. (New in 2.2)

COMMON FAILURES

- A sticky cookie banner covers the input field that just received focus.
- A sticky header hides the top of a long form as the user tabs downward.

EXCEPTION

- When the obscuring content is something the user opened and can dismiss.

RECOMMENDED TECHNIQUES

- C43 — use scroll-padding so sticky headers don't cover the focused element
- SCR: scroll the focused element into an unobscured view on focus

Official: https://www.w3.org/WAI/WCAG22/Understanding/focus-not-obscured-minimum

OPERABLE · GUIDELINE 2.4 · NAVIGABLE · EXAMPLE

2.4.11 Focus Not Obscured (Minimum)

LEVEL AA

THE SCENARIO

A tester tabs down a long signup form. A sticky promo banner pinned to the bottom of the screen covers the last two fields completely once focus reaches them.

✕ BEFORE — FAILS 2.4.11

```
<div class="sticky-banner">
  Get 20% off!</div>
// Covers focused fields below it

// Screen reader hears:
// "N/A — focused field is visually hidden"
```

✓ AFTER — PASSES 2.4.11

```
<style>html{
  scroll-padding-bottom:80px
}</style>

// Screen reader hears:
// "Focused field scrolls clear of the sticky banner"
```

Official: https://www.w3.org/WAI/WCAG22/Understanding/focus-not-obscured-minimum

2.5.1 Pointer Gestures

LEVEL A

Multipoint or path-based gestures must have a single-pointer alternative.

COMMON FAILURES

- A carousel that only responds to swipe gestures, with no buttons at all.
- A map that only zooms via pinch, with no visible + / − controls.

EXCEPTION

- When a multipoint or path-based gesture is essential, e.g. signature capture.

RECOMMENDED TECHNIQUES

- G215 — provide a single-pointer alternative for multipoint or path gestures
- SCR: add tap-based prev/next controls alongside swipe handlers

Official: https://www.w3.org/WAI/WCAG22/Understanding/pointer-gestures

2.5.1 Pointer Gestures

LEVEL A

THE SCENARIO

A product gallery only advances on swipe. A tester using a mouse (no touchscreen, tremor prevents precise dragging) has no way to move to the next photo at all.

✕ BEFORE — FAILS 2.5.1

```
<div ontouchstart="startSwipe()"
  ontouchend="endSwipe()">
// No tap/click alternative

// Screen reader hears:
// "No way to advance without a touch swipe"
```

✓ AFTER — PASSES 2.5.1

```
<div ontouchstart="startSwipe()">
  <button aria-label="Next photo"
    onclick="next()">›</button>
</div>

// Screen reader hears:
// "'Next photo, button' works with any pointer"
```

Official: https://www.w3.org/WAI/WCAG22/Understanding/pointer-gestures

2.5.2 Pointer Cancellation

LEVEL A

Down-event alone must not trigger actions; allow abort/undo by moving away before release.

COMMON FAILURES

- A button fires its action on mousedown, so there's no way to slide off and cancel a mis-tap.
- A drag-to-delete action that completes immediately on touch-down, before the drag even starts.

EXCEPTION

- When completion on the down-event is essential, e.g. a piano key or a game.

RECOMMENDED TECHNIQUES

- G210 — fire an action on the up-event, not the down-event
- F102 — avoid activating a control on down-event without an abort mechanism

Official: https://www.w3.org/WAI/WCAG22/Understanding/pointer-cancellation

2.5.2 Pointer Cancellation

LEVEL A

THE SCENARIO

A user with a hand tremor accidentally brushes a “Delete Account” button. Because it fires on mousedown, the action completes before they can move their pointer away to cancel it.

✕ BEFORE — FAILS 2.5.2

```
<button onmousedown=
  "deleteAccount()">Delete</button>

// Screen reader hears:
// "Action completes instantly on touch-down"
```

✓ AFTER — PASSES 2.5.2

```
<button onclick=
  "confirmDelete()">Delete</button>
// Fires on up-event, with confirm step

// Screen reader hears:
// "Action fires on release, after a confirm step"
```

Official: https://www.w3.org/WAI/WCAG22/Understanding/pointer-cancellation

2.5.3 Label in Name

LEVEL A

A control's accessible name must contain its visible label text.

COMMON FAILURES

- A button shows “Search” but has aria-label="submit query" — voice users saying “click Search” fail.
- An icon button labeled visually “Save” but with an accessible name of just “button”.

EXCEPTION

- None — the visible label text must be part of the accessible name.

RECOMMENDED TECHNIQUES

- G208 — include the visible text label as part of the accessible name
- H91 — make sure form controls have a name that includes the visible label

Official: https://www.w3.org/WAI/WCAG22/Understanding/label-in-name

2.5.3 Label in Name

LEVEL A

THE SCENARIO

A voice-control user says “Click Search” to activate the search button. Nothing happens — the button's accessible name is “submit query,” which doesn't match what's on screen.

✕ BEFORE — FAILS 2.5.3

```
<button aria-label=
  "submit query">Search</button>

// Screen reader hears:
// "'Click Search' doesn't match the accessible name"
```

✓ AFTER — PASSES 2.5.3

```
<button aria-label=
  "Search products">Search</button>

// Screen reader hears:
// "Visible 'Search' is part of the accessible name"
```

Official: https://www.w3.org/WAI/WCAG22/Understanding/label-in-name

2.5.4 Motion Actuation

LEVEL A

Functions triggered by device motion must also work via UI controls and be disableable.

COMMON FAILURES

- “Shake to undo” with no button alternative — unusable if the device is mounted.
- A tilt-to-scroll feature with no way to turn it off.

EXCEPTION

- When motion is essential, or it drives an accessibility-supported interface.

RECOMMENDED TECHNIQUES

- G213 — provide a conventional control alongside any motion-triggered action
- F108 — avoid restricting a function only to accelerometer/motion input

Official: https://www.w3.org/WAI/WCAG22/Understanding/motion-actuation

2.5.4 Motion Actuation

LEVEL A

THE SCENARIO

A note-taking app supports “shake to undo.” A user with a wheelchair-mounted tablet can't physically shake the device and has no other way to trigger undo.

✕ BEFORE — FAILS 2.5.4

```
window.addEventListener(
  'devicemotion', undo);
// No button alternative

// Screen reader hears:
// "Only a physical shake can trigger undo"
```

✓ AFTER — PASSES 2.5.4

```
window.addEventListener(
  'devicemotion', undo);
<button aria-label="Undo"
  onclick="undo()">↶</button>

// Screen reader hears:
// "'Undo, button' works without any motion"
```

Official: https://www.w3.org/WAI/WCAG22/Understanding/motion-actuation

2.5.7 Dragging Movements

LEVEL AA

Any drag operation must have a single-pointer alternative that isn't a drag. (New in 2.2)

COMMON FAILURES

- A kanban board where cards can only be moved by dragging — no menu, no buttons.
- A slider control with no clickable track and no arrow-key alternative to dragging.

EXCEPTION

- When dragging is essential, e.g. free-form drawing.

RECOMMENDED TECHNIQUES

- SCR: add a “Move to” context menu alongside drag-and-drop reordering
- G219 — provide a non-drag alternative for drag-based operations

Official: https://www.w3.org/WAI/WCAG22/Understanding/dragging-movements

2.5.7 Dragging Movements

LEVEL AA

THE SCENARIO

A project board lets users drag cards between columns. A tester with limited fine motor control can pick up a card with switch access, but there's no way to drop it — dragging is the only path.

✕ BEFORE — FAILS 2.5.7

```
<div draggable="true"
  ondragstart="pickUp()">
// No non-drag alternative

// Screen reader hears:
// "Only a drag gesture can move the card"
```

✓ AFTER — PASSES 2.5.7

```
<div draggable="true">
  <button aria-label="Move to
    column…">:</button>
</div>

// Screen reader hears:
// "'Move to column, button' offers a tap alternative"
```

Official: https://www.w3.org/WAI/WCAG22/Understanding/dragging-movements

2.5.8 Target Size (Minimum)

LEVEL AA

Pointer targets are at least 24×24 CSS px, or have enough spacing. (New in 2.2)

COMMON FAILURES

- Tiny 16px close (×) icons packed tightly together, easy to mis-tap.
- A row of small icon buttons with no spacing, so adjacent taps overlap.

EXCEPTION

- Inline links in a sentence, an equivalent target elsewhere, or essential/user-agent-default targets.

RECOMMENDED TECHNIQUES

- G217 — ensure a minimum 24×24 CSS px target size, or equivalent spacing
- C42 — use padding/min-height to enlarge a small tap target without changing its icon size

Official: https://www.w3.org/WAI/WCAG22/Understanding/target-size-minimum

2.5.8 Target Size (Minimum)

LEVEL AA

THE SCENARIO

A notifications panel lists several dismiss (×) icons at 16px each, packed edge-to-edge. A tester with a motor impairment keeps dismissing the wrong notification.

✕ BEFORE — FAILS 2.5.8

```
<button style="width:16px;
  height:16px">×</button>

// Screen reader hears:
// "N/A — mis-taps are a motor/visual issue"
```

✓ AFTER — PASSES 2.5.8

```
<button style="width:24px;
  height:24px;padding:8px">×</button>

// Screen reader hears:
// "Larger 24px target reduces accidental dismissal"
```

Official: https://www.w3.org/WAI/WCAG22/Understanding/target-size-minimum

SPOT THE BUG · 30 SEC

Keyboard (2.1.1)

```
<div class="btn" onclick="buy()">
  Buy now
</div>

// no tabindex, no role,
// no keydown handler
```

What can't a keyboard user do here?

Answer

Reach or activate it. A div isn't focusable or operable. Use <button>, or add role, tabindex=0, and Enter/Space handlers.

PRINCIPLE 3

Understandable

Is the content clear, and the interface predictable and forgiving?

13 CRITERIA · A + AA

3.1.1 Language of Page

LEVEL A

The default human language of each page is set programmatically.

COMMON FAILURES

- <html> has no lang attribute, so the screen reader may guess the wrong pronunciation engine.
- A lang value that's invalid, like lang="english" instead of a real BCP-47 code.

EXCEPTION

- None — every page needs a valid default language set.

RECOMMENDED TECHNIQUES

- H57 — set a valid lang attribute on the html element
- G34 — use a validator to confirm the primary language is correctly identified

Official: https://www.w3.org/WAI/WCAG22/Understanding/language-of-page

3.1.1 Language of Page

LEVEL A

THE SCENARIO

A tester runs NVDA on a Spanish-language landing page whose <html> tag has no lang attribute. NVDA defaults to its English voice and mangles every Spanish word it reads.

✕ BEFORE — FAILS 3.1.1

```
<html>
// No lang attribute at all

// Screen reader hears:
// "Spanish text read with English pronunciation"
```

✓ AFTER — PASSES 3.1.1

```
<html lang="es">

// Screen reader hears:
// "Spanish text read with the correct Spanish voice"
```

Official: https://www.w3.org/WAI/WCAG22/Understanding/language-of-page

3.1.2 Language of Parts

LEVEL AA

Passages in a different language are marked with their own lang attribute.

COMMON FAILURES

- A French testimonial embedded in an English page is read aloud in an English accent, becoming gibberish.
- A German product name mispronounced because the surrounding page never marks its language shift.

EXCEPTION

- Proper names, technical terms, and words absorbed into the surrounding language.

RECOMMENDED TECHNIQUES

- H58 — add a lang attribute to elements whose language differs from the page
- G34 — validate correct identification of language changes within content

Official: https://www.w3.org/WAI/WCAG22/Understanding/language-of-parts

3.1.2 Language of Parts

LEVEL AA

THE SCENARIO

An English case-study page quotes a French customer testimonial verbatim in the body copy. NVDA, still in English mode, reads the French passage phonetically and it's unintelligible.

✕ BEFORE — FAILS 3.1.2

```
<p>"C'est un excellent
  produit."</p>
// No lang change marked

// Screen reader hears:
// "French text read with English pronunciation rules"
```

✓ AFTER — PASSES 3.1.2

```
<p lang="fr">"C'est un
  excellent produit."</p>

// Screen reader hears:
// "Screen reader switches voice/pronunciation for the quote"
```

Official: https://www.w3.org/WAI/WCAG22/Understanding/language-of-parts

3.2.1 On Focus

LEVEL A

Moving focus to a component must not trigger an unexpected change of context.

COMMON FAILURES

- Tabbing into a dropdown auto-submits the form before the user chose an option.
- Focus entering a field automatically navigates to a new page.

EXCEPTION

- None — simply opening a menu on focus is fine; navigating away on focus is not.

RECOMMENDED TECHNIQUES

- G107 — don't trigger a change of context just by receiving focus
- H32 — require an explicit submit action rather than auto-submitting

Official: https://www.w3.org/WAI/WCAG22/Understanding/on-focus

3.2.1 On Focus

LEVEL A

THE SCENARIO

A tester tabs into a country <select> on a settings page. The instant it receives focus — before any value is even chosen — the page navigates away to a "region selected" confirmation screen.

✕ BEFORE — FAILS 3.2.1

```
<select onfocus=
  "location.href='/next'">

// Screen reader hears:
// "Page navigates away the instant focus lands"
```

✓ AFTER — PASSES 3.2.1

```
<select onchange=
  "confirmChange()">
// Change requires a value + submit

// Screen reader hears:
// "Focusing the field alone changes nothing"
```

Official: https://www.w3.org/WAI/WCAG22/Understanding/on-focus

3.2.2 On Input

LEVEL A

Changing a setting must not auto-trigger a context change unless the user was warned.

COMMON FAILURES

- Selecting a value in a <select> immediately reloads the page with no warning or submit step.
- Typing the last digit of an OTP field auto-advances focus in a way that surprises users.

EXCEPTION

- None, but users may be warned in advance that a change will happen.

RECOMMENDED TECHNIQUES

- G80 — require confirmation to minimize the chance of an unexpected context change
- H84 — use a submit button rather than auto-submitting a select on change

Official: https://www.w3.org/WAI/WCAG22/Understanding/on-input

3.2.2 On Input

LEVEL A

THE SCENARIO

A tester changes the language dropdown on a settings page and the page instantly reloads in the new language with no confirmation — losing their scroll position and any unsaved edits nearby.

✕ BEFORE — FAILS 3.2.2

```
<select onchange=
  "form.submit()">

// Screen reader hears:
// "Page reloads instantly on selection, no warning"
```

✓ AFTER — PASSES 3.2.2

```
<select onchange=
  "enableApplyButton()">
<button type="submit">Apply</button>

// Screen reader hears:
// "'Apply, button' becomes enabled; user confirms"
```

Official: https://www.w3.org/WAI/WCAG22/Understanding/on-input

3.2.3 Consistent Navigation

LEVEL AA

Repeated navigation appears in the same relative order across pages.

COMMON FAILURES

- The main nav reorders its links from page to page, disorienting returning users.
- A footer with links in a different order on every template across the site.

EXCEPTION

- When the user themselves initiates the change, e.g. personalization.

RECOMMENDED TECHNIQUES

- G61 — present repeated navigation components in the same relative order
- SCR: pull shared navigation from a single component to guarantee consistent order

Official: https://www.w3.org/WAI/WCAG22/Understanding/consistent-navigation

3.2.3 Consistent Navigation

LEVEL AA

THE SCENARIO

A user with cognitive fatigue relies on the nav bar being in the same order every time to build muscle memory for where “Billing” is. On the support subsite, the order is scrambled.

✕ BEFORE — FAILS 3.2.3

```
<!-- support-site nav.html -->
<nav>Billing, Home, Docs</nav>
// Different order than main site

// Screen reader hears:
// "Nav order changes unpredictably between sites"
```

✓ AFTER — PASSES 3.2.3

```
<!-- shared Nav component -->
<nav>Home, Docs, Billing</nav>
// Same order, every subsite

// Screen reader hears:
// "Nav order stays consistent everywhere"
```

Official: https://www.w3.org/WAI/WCAG22/Understanding/consistent-navigation

3.2.4 Consistent Identification

LEVEL AA

Components with the same function are labeled consistently across the site.

COMMON FAILURES

- “Search” on one page and “Find” (same icon, same function) on another confuse users.
- The same trash-can icon meaning “delete” on one screen and “archive” on another.

EXCEPTION

- None — components with the same function need consistent naming.

RECOMMENDED TECHNIQUES

- G197 — use consistent labels for components with the same functionality
- SCR: centralize icon/label choices in a shared component library

Official: https://www.w3.org/WAI/WCAG22/Understanding/consistent-identification

3.2.4 Consistent Identification

LEVEL AA

THE SCENARIO

A screen-reader user learns that the magnifying-glass icon means “Search” on the homepage. On the docs subsite, the identical icon is labeled “Find,” and they assume it's a different feature.

✕ BEFORE — FAILS 3.2.4

```
<!-- docs subsite -->
<button aria-label="Find">
  <svg class="magnifier"/></button>

// Screen reader hears:
// "'Find' on one site, 'Search' on another — same icon"
```

✓ AFTER — PASSES 3.2.4

```
<!-- shared across all subsites -->
<button aria-label="Search">
  <svg class="magnifier"/></button>

// Screen reader hears:
// "'Search' announced consistently everywhere"
```

Official: https://www.w3.org/WAI/WCAG22/Understanding/consistent-identification

3.2.6 Consistent Help

LEVEL A

If help (contact, chat, FAQ link) repeats across pages, it appears in the same relative order. (New in 2.2)

COMMON FAILURES

- A help link sits in the header on one page and is buried in the footer on the next.
- A live-chat launcher that appears bottom-right on most pages but top-left on checkout.

EXCEPTION

- Applies only to help that already exists on multiple pages — it doesn't require adding help.

RECOMMENDED TECHNIQUES

- SCR: place shared help/contact affordances in a single layout component
- G220 — keep help mechanisms in the same relative order/location across pages

Official: https://www.w3.org/WAI/WCAG22/Understanding/consistent-help

3.2.6 Consistent Help

LEVEL A

THE SCENARIO

A user with anxiety relies on knowing exactly where the “Live chat” bubble will be. On the checkout page it's been moved to a different corner by a different team, and they can't find it when they need it most.

✕ BEFORE — FAILS 3.2.6

```
<!-- checkout page only -->
<div class="chat" style=
  "top:0;left:0">Chat</div>

// Screen reader hears:
// "Help widget location varies unpredictably"
```

✓ AFTER — PASSES 3.2.6

```
<!-- shared layout, all pages -->
<div class="chat" style=
  "bottom:0;right:0">Chat</div>

// Screen reader hears:
// "Help widget stays in the same spot everywhere"
```

Official: https://www.w3.org/WAI/WCAG22/Understanding/consistent-help

3.3.1 Error Identification

LEVEL A

Input errors are detected and described to the user in text.

COMMON FAILURES

- A form turns invalid fields red but never states what's wrong in text.
- An error icon appears with no accompanying message a screen reader can announce.

EXCEPTION

- None — errors must be identified and described in text.

RECOMMENDED TECHNIQUES

- G83 — provide a text description when a required field or format error is detected
- ARIA21 — use aria-invalid and aria-describedby to expose errors to assistive tech

Official: https://www.w3.org/WAI/WCAG22/Understanding/error-identification

3.3.1 Error Identification

LEVEL A

THE SCENARIO

A tester submits a signup form with an invalid email. The field border turns red, but there's no error text — sighted users infer the problem from color, screen-reader users hear nothing.

✕ BEFORE — FAILS 3.3.1

```
<input class="error"
  style="border-color:red">
// No text error message

// Screen reader hears:
// "'Edit text, invalid' — no reason given"
```

✓ AFTER — PASSES 3.3.1

```
<input aria-invalid="true"
  aria-describedby="e1">
<span id="e1">Enter a valid email</span>

// Screen reader hears:
// "'Edit text, invalid, Enter a valid email'"
```

Official: https://www.w3.org/WAI/WCAG22/Understanding/error-identification

3.3.2 Labels or Instructions

LEVEL A

Inputs that need user input have labels or instructions.

COMMON FAILURES

- A field with only a placeholder “MM/DD/YYYY” that disappears the moment you start typing.
- A password field with hidden complexity rules revealed only after a failed submit.

EXCEPTION

- None — inputs that need user input must have labels or instructions.

RECOMMENDED TECHNIQUES

- G131 — provide descriptive labels or instructions for form controls
- G89 — provide expected data format and constraints up front, not just on error

Official: https://www.w3.org/WAI/WCAG22/Understanding/labels-or-instructions

3.3.2 Labels or Instructions

LEVEL A

THE SCENARIO

A tester clicks into a date-of-birth field. The only guidance was a placeholder “MM/DD/YYYY” that vanished the moment they typed the first digit — no persistent label anywhere.

✕ BEFORE — FAILS 3.3.2

```
<input placeholder=
  "MM/DD/YYYY">
// No <label>, placeholder only

// Screen reader hears:
// "'Edit text' — format hint disappears on typing"
```

✓ AFTER — PASSES 3.3.2

```
<label for="dob">
  Date of birth (MM/DD/YYYY)</label>
<input id="dob">

// Screen reader hears:
// "'Date of birth (MM/DD/YYYY), edit text'"
```

Official: https://www.w3.org/WAI/WCAG22/Understanding/labels-or-instructions

3.3.3 Error Suggestion

LEVEL AA

If an error is detected and a correction is known, suggest it (unless it risks security).

COMMON FAILURES

- “Invalid date” with no hint about the expected format or valid range.
- “Password rejected” with no mention of which rule actually failed.

EXCEPTION

- When suggesting a fix would jeopardize the security or purpose of the content.

RECOMMENDED TECHNIQUES

- G177 — offer a suggestion for fixing an input error when one is known
- G85 — provide a specific error message identifying the constraint that was violated

Official: https://www.w3.org/WAI/WCAG22/Understanding/error-suggestion

3.3.3 Error Suggestion

LEVEL AA

THE SCENARIO

A tester enters “13/40/2024” in a date field and gets “Invalid date.” Nothing tells them the expected format is DD/MM/YYYY or that day 40 doesn't exist.

✕ BEFORE — FAILS 3.3.3

```
<span class="error">
  Invalid date</span>

// Screen reader hears:
// "'Invalid date' — no idea what to fix"
```

✓ AFTER — PASSES 3.3.3

```
<span class="error">
  Use DD/MM/YYYY, e.g. 04/07/2024
</span>

// Screen reader hears:
// "'Use DD/MM/YYYY, e.g. 04/07/2024'"
```

Official: https://www.w3.org/WAI/WCAG22/Understanding/error-suggestion

3.3.4 Error Prevention (Legal, Financial, Data)

LEVEL AA

For legal/financial/data submissions: make them reversible, checked, or confirmable.

COMMON FAILURES

- A one-click “Transfer funds” button with no review step and no undo.
- A “Delete all data” action that executes instantly on a single click.

EXCEPTION

- None for the covered transaction types — legal, financial, and data-modifying actions.

RECOMMENDED TECHNIQUES

- G164 — provide a way to reverse a submitted transaction
- G98 — provide a review step before final submission of a legal/financial action

Official: https://www.w3.org/WAI/WCAG22/Understanding/error-prevention-legal-financial-data

3.3.4 Error Prevention (Legal, Financial, Data)

LEVEL AA

THE SCENARIO

A tester clicks “Transfer $500” on a banking demo and the money moves instantly — no confirmation screen, no summary of the recipient and amount, no way to undo a fat-finger mistake.

✕ BEFORE — FAILS 3.3.4

```
<button onclick=
  "transfer(500)">Transfer</button>

// Screen reader hears:
// "Funds move immediately with no review"
```

✓ AFTER — PASSES 3.3.4

```
<button onclick=
  "showReview(500)">Transfer</button>
// Opens a review-and-confirm step

// Screen reader hears:
// "'Review transfer of $500, confirm?' appears first"
```

Official: https://www.w3.org/WAI/WCAG22/Understanding/error-prevention-legal-financial-data

3.3.7 Redundant Entry

LEVEL A

Don't make users re-enter info they already gave in the same process. (New in 2.2)

COMMON FAILURES

- A checkout flow asks for the shipping address again as billing with no “same as” option.
- A multi-step signup form that forgets the email you entered on step 1 by step 3.

EXCEPTION

- When re-entry is essential, e.g. confirming a password or a genuine memory test.

RECOMMENDED TECHNIQUES

- SCR: auto-populate previously entered values, or offer a copy checkbox
- G: minimize re-entry by carrying form state across steps in the same process

Official: https://www.w3.org/WAI/WCAG22/Understanding/redundant-entry

3.3.7 Redundant Entry

LEVEL A

THE SCENARIO

A tester fills in a full shipping address, then reaches the billing-address step and has to retype the identical address by hand — there's no “same as shipping” checkbox anywhere.

✕ BEFORE — FAILS 3.3.7

```
<h3>Billing Address</h3>
<input placeholder="Street">
// No reuse of shipping address

// Screen reader hears:
// "Full address must be retyped from scratch"
```

✓ AFTER — PASSES 3.3.7

```
<h3>Billing Address</h3>
<label><input type="checkbox">
  Same as shipping</label>

// Screen reader hears:
// "'Same as shipping, checkbox' auto-fills the form"
```

Official: https://www.w3.org/WAI/WCAG22/Understanding/redundant-entry

3.3.8 Accessible Authentication (Minimum)

LEVEL AA

Don't require a cognitive function test (memory, puzzle, transcription) to log in. (New in 2.2)

COMMON FAILURES

- A login that forces solving a puzzle CAPTCHA before signing in.
- A password field that blocks paste, forcing users to retype it from memory.

EXCEPTION

- Object-recognition and personal-content CAPTCHAs are allowed alternatives.

RECOMMENDED TECHNIQUES

- G218 — allow password managers and pasting into authentication fields
- SCR: avoid cognitive tests (puzzles, transcription) as the only login path

Official: https://www.w3.org/WAI/WCAG22/Understanding/accessible-authentication-minimum

3.3.8 Accessible Authentication (Minimum)

LEVEL AA

THE SCENARIO

A user with a cognitive disability relies on a password manager to log in. The login form blocks paste on the password field, forcing them to manually retype a 20-character random password from memory.

✕ BEFORE — FAILS 3.3.8

```
<input type="password"
  onpaste="return false">

// Screen reader hears:
// "Paste is blocked; password must be retyped"
```

✓ AFTER — PASSES 3.3.8

```
<input type="password">
// Paste and autofill both allowed

// Screen reader hears:
// "Password manager autofill and paste both work"
```

Official: https://www.w3.org/WAI/WCAG22/Understanding/accessible-authentication-minimum

PRINCIPLE 4

Robust

Will it keep working across browsers and assistive tech, now and later?

2 CRITERIA · A + AA

ROBUST · GUIDELINE 4.1 · COMPATIBLE

4.1.2 Name, Role, Value

LEVEL A

For all UI components, name + role + state/value must be programmatically determinable and settable.

COMMON FAILURES

- A styled as a checkbox with no role, no name, and no checked state exposed.
- A custom slider whose value never updates in the accessibility tree as it changes.

EXCEPTION

- None — name, role, and value must be programmatically exposed for every component.

RECOMMENDED TECHNIQUES

- ARIA14 / ARIA16 — expose an accessible name via aria-label or aria-labelledby
- ARIA4 — keep a custom widget's state (checked, expanded) in sync via ARIA

Official: https://www.w3.org/WAI/WCAG22/Understanding/name-role-value

4.1.2 Name, Role, Value

LEVEL A

THE SCENARIO

A tester builds a custom checkbox from a styled with a click handler. NVDA reports it only as “clickable” — no role, no name, and toggling it never announces a state change.

✕ BEFORE — FAILS 4.1.2

```
<div class="checkbox"
  onclick="toggle()"></div>

// Screen reader hears:
// "'Clickable' — no role, name, or state"
```

✓ AFTER — PASSES 4.1.2

```
<div class="checkbox" role="checkbox"
  aria-checked="false"
  tabindex="0" onclick="toggle()">
</div>

// Screen reader hears:
// "'Remember me, checkbox, not checked'"
```

Official: https://www.w3.org/WAI/WCAG22/Understanding/name-role-value

ROBUST · GUIDELINE 4.1 · COMPATIBLE

4.1.3 Status Messages

LEVEL AA

Status messages can be announced by assistive tech without moving focus.

COMMON FAILURES

- “3 results found” appears silently on screen with no live region announcing it.
- “Item added to cart” flashes visually but a screen-reader user never hears it.

EXCEPTION

- None — status messages must be announced without requiring a focus move.

RECOMMENDED TECHNIQUES

- ARIA19 — use role="alert" or role="status" to expose important status changes
- G199 — announce dynamic status changes without moving keyboard focus

Official: https://www.w3.org/WAI/WCAG22/Understanding/status-messages

ROBUST · GUIDELINE 4.1 · COMPATIBLE · EXAMPLE

4.1.3 Status Messages

LEVEL AA

THE SCENARIO

A tester searches a product catalog. The result count updates from “120 results” to “3 results” visually as they type, but NVDA never announces the change — there's no live region.

✕ BEFORE — FAILS 4.1.3

```
<div id="count">
  120 results</div>
// Plain div, not a live region

// Screen reader hears:
// "Result count changes silently on screen"
```

✓ AFTER — PASSES 4.1.3

```
<div id="count" role="status"
  aria-live="polite">
  3 results</div>

// Screen reader hears:
// "'3 results' is announced automatically"
```

Official: https://www.w3.org/WAI/WCAG22/Understanding/status-messages

04

THE PROCESS

How to actually test

Three passes. Each catches what the others can't — skip one and you ship the gap.

TESTING PROCESS

Three layers, no shortcuts

Automated

~30–40%

Fast, repeatable, CI-friendly. Catches contrast, missing alt/labels, ARIA misuse, invalid attributes.

Blind spot: Can't judge meaning: is the alt good? Is focus order logical?

Manual

~adds 50%

Human judgement. Keyboard-only walkthrough, zoom/reflow, focus order, link clarity, error handling.

Blind spot: Slower; needs a checklist so nothing is skipped.

Screen reader

the last mile

Real AT experience. Confirms name/role/state, announcements, live regions, reading order.

Blind spot: Requires practice; pair SR with its best browser.

TESTING PROCESS

Which layer catches what

Criterion	Automated	Manual	Screen reader
Color contrast (1.4.3)	● Auto	○	○
Missing alt / labels (1.1.1, 3.3.2)	● Auto	◑ verify	● confirm
Alt is meaningful (1.1.1)	—	● Manual	● SR
Keyboard operability (2.1.1)	◑ partial	● Manual	○
Focus order & visible focus (2.4.3/.7)	○	● Manual	◑
Reflow & zoom (1.4.10/.4)	○	● Manual	○
Name / Role / State (4.1.2)	◑ partial	◑	● SR
Live region announcements (4.1.3)	—	○	● SR

● full · ◑ partial · ○ weak / supporting · — not covered. Notice: no single column is all ●.

KEYBOARD · DEEP DIVE

tabindex — the three values that matter

`tabindex="0"`

USE FREELY

Adds a custom element (a div widget) into the natural tab order, in DOM position. The correct way to make non-native controls focusable.

`tabindex="-1"`

USE DELIBERATELY

Focusable by script (element.focus()) but skipped by Tab. Perfect for moving focus to a modal, error summary, or new view.

`tabindex="1+"`

ALMOST NEVER

Positive values jump the element to the front of the tab order, breaking the natural sequence everywhere. An anti-pattern.

Golden rule: prefer native elements (button, a, input) — they're keyboard-ready with no tabindex at all. Reach for tabindex only on custom widgets.

05

ARIA

Power tool, sharp edge

It can describe anything to a screen reader — including the wrong thing. Handle with care.

ARIA

What it is — and what it isn't

Accessible Rich Internet Applications

A set of HTML attributes that supply name, role, state, and relationships to assistive tech — for cases native HTML can't express.

Roles — what it is (role="tab")

States/Properties — its condition (aria-selected)

Relationships — links elements (aria-controls)

ARIA changes only how AT perceives an element. It adds NO behavior, NO focus, NO keyboard handling.

THE SAME WIDGET, THREE WAYS

```
// 1. Native — best
<button>Menu</button>

// 2. ARIA where HTML can't
<div role="tab"
     aria-selected="true"
     tabindex="0">Home</div>

// 3. Redundant — don't
<button role="button">Go</button>
```

ARIA

Role + state + relationship, in one widget

```
<button id="acc-btn"
        aria-expanded="false"
        aria-controls="panel">
  Shipping details
</button>
<div id="panel" role="region"
     aria-labelledby="acc-btn"
     hidden>…</div>
```

Attribute	Type	What it does
`role="region"`	**Role**	Names a landmark/section
`aria-expanded`	**State**	Open/closed — must update on toggle
`aria-controls`	**Relationship**	Points button → the panel it opens
`aria-labelledby`	**Property**	Borrows the button's text as the name

Critical: aria-expanded must flip to "true" in JS when the panel opens. A state that never changes is worse than none — it lies to the user.

ARIA

The 5 rules of ARIA

Don't use ARIA if native HTML can do it	A <button> beats every time — you get focus, keyboard, and state for free.
Don't change native semantics	Never <h2 role="button">. Wrap a real control instead: <button>…</button> .
All interactive ARIA must be keyboard-operable	If you add role="button", you own Tab focus + Enter/Space handling.
Don't hide focusable elements	Never put role="presentation" or aria-hidden="true" on a focusable element — you create a silent tab stop.
Interactive elements need an accessible name	Every control must have a name via label, text, aria-label, or aria-labelledby.

Paraphrased from the W3C "Using ARIA" rules. Source: w3.org/TR/using-aria

When to reach for it — and when not to

Use ARIA when…

- Custom widgets HTML can't express (tabs, trees, comboboxes)
- Live regions for dynamic updates (4.1.3)
- Naming landmarks & grouping (aria-labelledby)
- Expressing state native HTML lacks (aria-expanded, aria-current)
- Wiring relationships (aria-describedby for hints/errors)

Skip ARIA when…

- A native element already does the job
- You haven't added the keyboard behavior to match
- Just to silence an automated-tool warning
- On content that should simply be real text/HTML
- Guessing — “No ARIA is better than bad ARIA”

ARIA

role="presentation" vs. aria-hidden="true"

They sound similar and are constantly confused. The difference is what disappears.

`role="presentation"`

(= role="none")

Strips the element's semantics but KEEPS its children and text in the tree.

The element's role is ignored; content is still read.

Use it for: A layout <table> → role="presentation" so it's not announced as a data table, but the cell text still reads.

`aria-hidden="true"`

(removes the subtree)

Removes the element AND all descendants from the accessibility tree entirely.

Nothing inside is exposed to AT — but it stays visible on screen.

Use it for: A decorative icon next to visible text → aria-hidden so it isn't read twice.

Trap: aria-hidden="true" on anything focusable hides it from AT but keeps it tabbable — a silent, nameless tab stop.

ARIA

role="application" — the nuclear option

What it does

It tells the screen reader: “stop browse mode — pass every keystroke straight to this app.” All built-in reading shortcuts (H, K, arrows) are surrendered to your JavaScript.

So YOU must now implement every interaction the screen reader normally provides — or the user is stranded.

Rare, valid uses

- Fully custom canvas apps (a web-based spreadsheet, an online IDE, a game)
- Widgets with their own complete keyboard model and focus management

Don't use it for

- Ordinary pages, forms, articles, or content
- A single widget when role="tablist"/"menu" etc. fits
- “Making it feel app-like” — it breaks SR navigation for everyone

06

LIVE REGIONS

Announcing change

How a screen reader user hears “cart updated” without losing their place.

LIVE REGIONS

Speaking up without stealing focus

A live region is a container AT watches; when its contents change, the update is announced automatically — no focus move needed. This is how you satisfy 4.1.3 Status Messages.

aria-live	`polite | assertive | off`	polite = wait for a pause; assertive = interrupt now; off = ignore.
aria-atomic	`true | false`	true = re-read the WHOLE region; false (default) = read only what changed.
aria-relevant	`additions | removals | text | all`	Which kinds of change get announced. Default: additions text.

Shortcut: role="status" ≈ aria-live="polite" + aria-atomic="true". role="alert" ≈ aria-live="assertive".

LIVE REGIONS

Recipes — pick the right values

Form validation summary

`alert + assertive`

Errors must interrupt. role="alert" (assertive, atomic) read in full immediately.

“Item added to cart” toast

`status + polite`

Non-urgent. role="status", polite, atomic=true so the whole message reads.

Search results count

`polite + atomic=true`

“12 results” should read as one unit after typing stops, not letter by letter.

Chat / activity feed

`log, polite, relevant=additions`

Announce only new messages as they arrive; don't re-read history.

Progress / % complete

`polite, atomic=true (throttle)`

Announce milestones, not every tick — or you flood the user.

Timer / clock

`aria-live="off"`

Constant change would never stop talking. Keep it silent; announce on demand.

Gotcha: the live region must already exist in the DOM before you inject text. Adding the node and its content at the same time often goes unannounced.

07

SHIP IT WELL

Reporting, AI & tooling

A bug nobody can reproduce gets ignored. Write the report that gets fixed.

REPORTING

Anatomy of an a11y bug report

1. **Page / Module** Exact URL + component, e.g. “Checkout · Payment form”
2. **WCAG criterion** Number + level: “1.4.3 Contrast (Minimum) — AA”. Cite, don't opine.
3. **Issue summary** One scannable line: “Error text fails 4.5:1 contrast (2.9:1)”
4. **Description** User impact + how to reproduce, with the AT/browser used.
5. **Occurrences** Combine identical issues across the page into one ticket, listed.
6. **Source code** The offending snippet — the dev shouldn't have to hunt.
7. **Recommendation** A concrete fix or code suggestion, not just “it's broken.”
8. **Screenshot / recording** Visual proof, or a short SR audio/video clip where useful.

REPORTING

A filled-in example

PAGE / MODULE

Checkout · Payment form

WCAG

4.1.2 Name, Role, Value — Level A

SUMMARY

Card-type radio group has no accessible name

DESCRIPTION

NVDA + Chrome announces “radio button” with no group label, so users can't tell what they're choosing. Repro: Tab into the card-type options.

OCCURRENCES

3 × on this page (card type, save-card, billing same-as)

SOURCE

```
<div>
  <input type="radio" name="ct">Visa
  <input type="radio" name="ct">Amex
</div>
```

RECOMMENDATION

```
<fieldset>
  <legend>Card type</legend>
  <label><input type="radio"
    name="ct"> Visa</label>
  …
</fieldset>
```

Screenshot / SR recording attached →

TOOLING

Use AI to go faster — not to skip judgement

Great accelerators

- Draft alt text & ARIA labels for review
- Explain why a criterion failed in plain English
- Generate test cases & edge-case checklists
- Suggest fixes / refactors for flagged code
- Summarize an axe scan into a triaged report
- Turn a manual finding into a clean ticket

Keep the human in the loop

- AI can't confirm alt is meaningful — you must
- It will hallucinate “WCAG passes” — verify
- It can't feel a confusing SR experience
- Don't paste private user data into prompts
- Auto-“fixes” can add wrong ARIA — re-test

SHIFT LEFT

Accessibility design annotations

Catch issues in Figma, before a line of code. Annotations tell devs the a11y intent that visuals alone can't carry.

Heading order

Mark H1–H3 levels so structure isn't guessed from font size

Focus / tab order

Number the path keyboard focus should follow

Names & labels

Specify accessible names for icon-only controls

Live regions

Flag what gets announced on change (toasts, counts)

Alt text

Provide alt copy, or mark images decorative

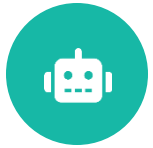
States

Define focus, hover, error, disabled visuals + contrast

Tools: Figma “A11y Annotation Kit” / Stark / axe for Designers. Annotate once, save the team a dozen tickets.

TOOLBOX

What we test with



Automated scanners

- axe DevTools (browser ext + CI)
- WAVE (WebAIM)
- Lighthouse a11y audit
- Pa11y / axe-core in pipelines
- Accessibility Insights

Manual & inspect

- Keyboard only (Tab/Shift+Tab)
- Browser DevTools → Accessibility tree
- Contrast: TPGi CCA, WebAIM checker
- Screen readers: NVDA, JAWS, VoiceOver
- Zoom 200% / reflow at 320px

Bookmarklets & extras

- Text-spacing & 1.4.12 bookmarklet
- Headings / landmarks (HeadingsMap)
- ARIA-DevTools, Accessibility Insights tab-stops
- tota11y · ANDI
- PEAT for flash thresholds

MAKE IT YOURS

Checklist & reference hub

Our team checklist

- Per-component gates: links, buttons, forms, modals, tables, media
- Keyboard pass · focus visible · focus order · no traps
- Contrast · reflow · 200% zoom · text spacing
- SR smoke test: name / role / state / announcements
- **[Link our living checklist doc here]**

Reference links

WCAG 2.2 spec — w3.org/TR/WCAG22/

How to Meet (Quickref) — w3.org/WAI/WCAG22/quickref/

ARIA Authoring Practices — w3.org/WAI/ARIA/apg/

WebAIM articles — webaim.org/

Deque University / axe rules — dequeuniversity.com/rules/axe/

[Our internal a11y wiki] — add-link-here

THE WHOLE JOB, IN ONE LINE

Test it by keyboard.
Hear it on a screen reader.
Measure it against WCAG.

Then write the report that gets it fixed. Do that, and you're not just QA — you're the reason someone can use what we build.